

Project Structure & Code Walkthrough

Memory-Based Detection of Golden Ticket & SIDHistory Abuse in Active Directory

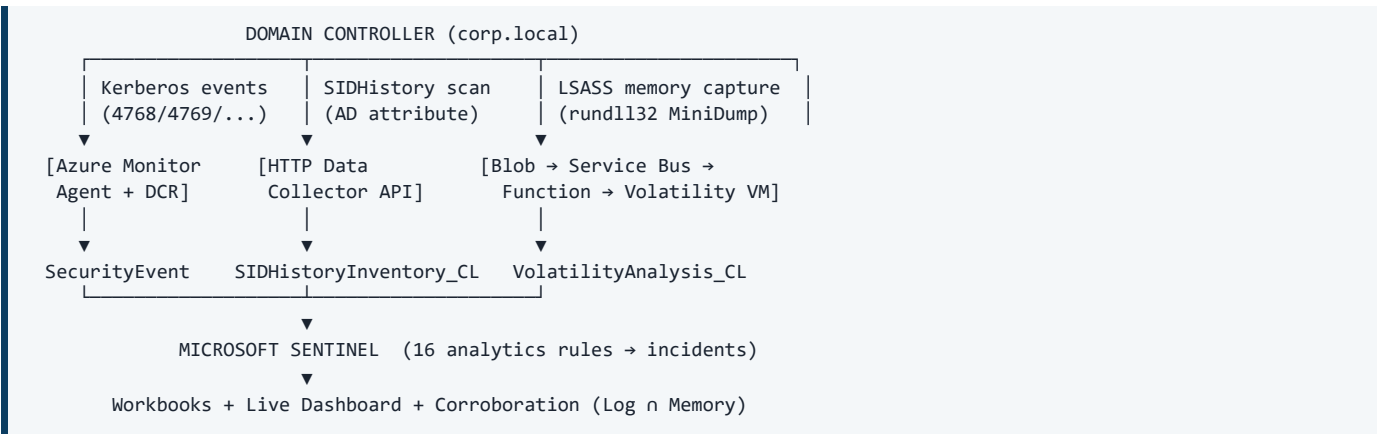
P Rahul (R23MTC09), REVA University — MTech Capstone. A guide to the whole codebase for review in VS Code.

How to open in VS Code

```
# clone from GitHub (or open the local folder)
git clone https://github.com/RAHUL202-coder/ad-golden-ticket-detection.git
cd ad-golden-ticket-detection
code . # opens the whole project in VS Code
```

Recommended VS Code extensions: **Python**, **PowerShell**, **Azure Account**, **Kusto (KQL)**.

The big picture — 3 detection paths into Microsoft Sentinel



Directory tree (annotated)

```
ad-golden-ticket-detection/
|
|-- README.md # Project overview (repo landing page)
|-- CLAUDE.md # Master reference: all IDs, config, live state
|-- PROJECT-ARCHITECTURE.md # Architecture write-up
|-- PROJECT-KNOWLEDGE.md # Accumulated project knowledge
|-- PROJECT-STRUCTURE.md # ← THIS FILE (code walkthrough)
|-- architecture-diagram.html # Visual architecture (open in browser)
|
|-- kql/ # ★ THE DETECTION LOGIC (Sentinel analytics rules)
|   |-- golden-ticket-ptt.kql # Golden Ticket: TGS without prior TGT
|   |-- golden-ticket-rc4-downgrade.kql # Golden Ticket: RC4 encryption downgrade
|   |-- sidhistory-high-risk.kql # SIDHistory: privileged SID injection
|   |-- volatility-memory-hit.kql # Memory: forged ticket recovered from LSASS
|
|-- scripts/ # ★ DOMAIN CONTROLLER-SIDE PowerShell
|   |-- Capture-LSASS.ps1 # Dumps LSASS memory → uploads to Blob (Managed Identity)
|   |-- SIDHistory-Inventory.ps1 # Scans AD for sIDHistory attribute (LDAP)
|   |-- SIDHistory-Inventory-Upload.ps1 # Posts findings to Log Analytics (HTTP Data Collector)
|   |-- Simple-SIDHistory-Upload.ps1 # Minimal SIDHistory uploader
|   |-- Infra-HealthCheck.ps1 # 19-point infrastructure health check
```

```

├── Close-AllIncidents.ps1      # Bulk-close Sentinel incidents (reset between demos)
├── Deploy-Change2-SentinelRules.ps1 # Deploys/updates the analytics rules
├── Deploy-Change3-LogicApp.ps1  # Deploys the response Logic App
├── function-app/               # ★ AZURE FUNCTIONS (the memory pipeline glue)
│   ├── function_app.py         # Function App entry point
│   ├── LsassDumpTrigger/      # Blob trigger: new .dmp → Service Bus queue
│   │   ├── __init__.py
│   │   └── function.json      # binding: blob artifacts/{name} → memory-dump-queue
│   ├── AnalysisResultProcessor/ # Queue trigger: result → VolatilityAnalysis_CL
│   │   ├── __init__.py
│   │   └── function.json      # binding: serviceBus analysis-queue → Log Analytics
│   ├── host.json              # Functions host config
│   └── requirements.txt        # Python dependencies
├── azure/                      # ★ AZURE RESOURCE DEFINITIONS & OPS
│   ├── volatility-worker/
│   │   ├── worker.py          # ★★ THE MEMORY-FORENSICS ENGINE (runs on the VM)
│   │   └── README.md          # deploy steps (systemd, pypykatz install)
│   ├── workbooks/
│   │   ├── capstone-evidence-workbook.json # Overview dashboard
│   │   └── attack-corroboration-workbook.json # Dual-track corroboration dashboard
│   ├── live-dashboard/
│   │   ├── SOC-Dashboard.html # Live cloud dashboard (hosted on Azure Storage)
│   │   └── dashboard-api-logicapp.json # Logic App that feeds it live data
│   ├── lab-control/
│   │   ├── Start-Lab.ps1 / STOP-LAB.cmd # Bring lab up/down (save credits)
│   │   └── README.md
│   └── Pause-Azure.ps1 / Resume-Azure.ps1 # (legacy) pause/resume scripts
├── attack-simulation/         # ★ ATTACK SCRIPTS (for validation/demo)
│   ├── Start-AttackSim.ps1    # Pre-attack setup + baseline
│   ├── Run-GoldenTicketAttack.ps1 # Golden Ticket (DCSync → forge → inject)
│   ├── Run-SIDHistoryAttack.ps1 # SIDHistory injection
│   └── End-AttackSim.ps1      # Teardown/cleanup
├── diagrams/                  # Architecture & pipeline diagrams (HTML/MD)
├── docs/                      # ★ REPORT & VIVA MATERIALS
│   ├── benchmark-results.md   # Measured P/R/F1 = 1.00 (40 samples)
│   ├── pypykatz-contribution.md # The novel memory-detection finding
│   ├── REPORT-ERRATA.md       # Corrections applied to the report
│   ├── Demo-Steps.pdf         # Live demo playbook
│   ├── Viva-QA-Notes.md       # Ready viva answers
│   ├── Video-Recording-Script.pdf # Screen-recording narration
│   └── Speaker-Notes.md       # Presentation notes

```

The 5 most important files (what a reviewer should read first)

1. `azure/volatility-worker/worker.py` — ★★ the novel contribution

Runs as a systemd service on the Volatility VM. Polls `memory-dump-queue`, downloads each LSASS `.dmp` from Blob, and **recovers Kerberos tickets from memory**. A forged TGT (service=krbtgt) for a *user* account with a **lifetime > 365 days** is flagged **CRITICAL** (mimikatz forges ~3650-day tickets; legitimate ≈ 10 hours). Posts the result to `analysis-queue`.

> Key functions: `extract_kerberos_pypykatz()`, `_kirbi_lifetime_days()`, `compute_risk()`.

2. `kql/golden-ticket-ptt.kql` — the primary detection

A TGS request (4769) with **no preceding TGT (4768)** = a pre-forged ticket. Implemented as a `leftanti` join (4769 accounts minus 4768 accounts). This is the workhorse rule.

3. `scripts/Capture-LSASS.ps1` — DC-side capture

Uses `rundll132 comsvcs.dll MiniDump` to dump `lsass.exe`, then uploads the `.dmp` to Blob Storage using the DC's **Managed Identity** (no hardcoded keys). This starts the memory pipeline.

4. `function-app/` — the pipeline glue

Two Azure Functions: **LsassDumpTrigger** (new blob → Service Bus queue) and **AnalysisResultProcessor** (analysis result → writes to `VolatilityAnalysis_CL`). These connect the DC, the VM, and Sentinel without any component being tightly coupled.

5. `azure/workbooks/attack-corroboration-workbook.json` — the thesis proof

The dual-track dashboard. Its core query is an **inner join**: log-track accounts (4769) n memory-track accounts (CRITICAL) → verdict **CONFIRMED — Log + Memory**.

How each file maps to the data flow

Pipeline stage	File that implements it
----- -----	
Attack (for testing)	<code>attack-simulation/Run-GoldenTicketAttack.ps1</code>
LSASS capture on DC	<code>scripts/Capture-LSASS.ps1</code>
Blob → queue	<code>function-app/LsassDumpTrigger/</code>
Memory analysis	<code>azure/volatility-worker/worker.py</code>
Result → Sentinel table	<code>function-app/AnalysisResultProcessor/</code>
Kerberos log detection	<code>kql/golden-ticket-*.kql</code>
SIDHistory detection	<code>scripts/SIDHistory-Inventory*.ps1</code> + <code>kql/sidhistory-high-risk.kql</code>
Memory detection rule	<code>kql/volatility-memory-hit.kql</code>
Corroboration	<code>azure/workbooks/attack-corroboration-workbook.json</code>
Visualisation	<code>azure/workbooks/*.json</code> + <code>azure/live-dashboard/SOC-Dashboard.html</code>
Ops (start/stop)	<code>azure/lab-control/</code>

Languages & technologies used

Layer	Technology
----- -----	
Detection logic	KQL (Kusto Query Language) — Microsoft Sentinel
DC-side automation	PowerShell
Memory forensics	Python (pypykatz, minikerberos) on Ubuntu VM
Pipeline glue	Azure Functions (Python)

Transport	Azure Blob Storage, Service Bus
Ingestion	Azure Monitor Agent + DCR , HTTP Data Collector API
Visualisation	Sentinel Workbooks (JSON) + HTML/JS dashboard
Cloud platform	Microsoft Azure + Azure Arc (on-prem DC)

How to run it (high level)

1. **Bring the lab up:** `azure/lab-control/START-LAB.cmd`
 2. **Run an attack:** `attack-simulation/Run-GoldenTicketAttack.ps1` (or the staged demo)
 3. **Watch detection:** Sentinel → Incidents; `VolatilityAnalysis_CL`; the corroboration workbook
 4. **Stop to save credits:** `azure/lab-control/STOP-LAB.cmd`
-

Honest notes (for accurate review)

- **Memory engine:** the ticket recovery uses **pypykatz** (Volatility 3 has no working Kerberos-ticket plugin on modern Windows). User-facing dashboards say "memory forensics" generically.
- **Benchmark:** measured **P/R/F1 = 1.00** applies to the **SIDHistory detector** (40 samples). The Golden Ticket path is validated per-case + by corroboration. No fabricated blanket accuracy is claimed.
- **RC4 rule** is deployed but does not fire in a single-DC lab (documented constraint).
- **Automated response** (Teams alert / auto-capture) is scaffolded; the outward actions need OAuth and are documented as future work. Detection, triage, and visualisation are fully automated.